Report

Social Media Feed

Names : Smagul Tumar , Kambarbek Zeine, Yermukan Zamira

Group: SE-2510

Published URL: https://github.com/smagultumar/final-sdp

INTRODUCTION :

This project is a Java console application that demonstrates design patterns in a social media feed. It supports text and image posts, publishing posts, notifying followers, sorting the feed, decorating post displays, and calculating statistics.

The aim of the project is to show how different design patterns can work together in one application. The project uses Builder, Abstract Factory, Factory Method, Bridge, Facade, Decorator, Strategy, Visitor, and Observer.

BODY :

Application structure :

The application is organized into Java packages. The model package contains posts and the feed. Other packages contain the classes for post creation, rendering, sorting, notifications, decorators, and visitors. The Main class starts the application.

The application workflow is:

1. A post is created with PostBuilder or PostFactory .

2. The post is published to Feed .

3. The feed notifies its observers.

4. A strategy sorts the posts.

5. The view displays posts using a renderer.

6. Decorators can add an emoji or hashtag.

7. A visitor calculates feed statistics.

Design patterns :

Builder : PostBuilder creates a post step by step.

Abstract Factory : SocialPlatformFactory creates a compatible renderer and notification sender.

Factory Method : PostFactory selects a factory for creating a text or image post.

Bridge : FeedView separates feed presentation from renderer implementations.

Facade : SocialMediaFacade provides simple methods for publishing and showing posts.

Decorator : EmojiDecorator and HashtagDecorator add display details to a post.

Strategy : NewestFirstStrategy and MostLikedStrategy provide different feed sorting algorithms.

Visitor : FeedStatisticsVisitor counts text posts, image posts, and likes.

Observer : Feed notifies subscribers when a new post is published**.**

JAVA CODE :

BUILDER :

PostBuilder.java

```java
package socialfeed.builder;

import socialfeed.model.*;
public class PostBuilder {  8 usages  tumar
    private String id;  4 usages
    private String author;  4 usages
    private String text = "";  3 usages
    private String imageUrl = "";  4 usages

    public PostBuilder setId(String id) { this.id = id; return this; }  2 usages  tumar
    public PostBuilder setAuthor(String author) { this.author = author; return this; }  2 usages  tumar
    public PostBuilder setText(String text) { this.text = text; return this; }  2 usages  tumar
    public PostBuilder setImageUrl(String imageUrl) { this.imageUrl = imageUrl; return this; }  no usages  tumar

    public Post build() {  2 usages  tumar
        if (id == null || author == null) throw new IllegalStateException("Post id and author are required");
        if (imageUrl != null && !imageUrl.isBlank()) return new ImagePost(id, author, text, imageUrl);
        return new TextPost(id, author, text);
    }
}

```

ABSTRACT FACTORY :

SocialPlatformFactory.java

```java
package socialfeed.factory;

import ...
public interface SocialPlatformFactory {  5 usages  2 implementations  tumar
    FeedRenderer createRenderer();  1 usage  2 implementations  tumar
    NotificationSender createNotificationSender();  1 usage  2 implementations  tumar
}

```

FACTORY METHOD :

PostFactory.java

```java
package socialfeed.factory;

import ...
public abstract class PostFactory {  7 usages  2 inheritors  tumar
    public abstract Post createPost(String id, String author, String text, String mediaUrl);

    public static PostFactory forType(String type) {  2 usages  tumar
        if ("TEXT".equalsIgnoreCase(type)) return new TextPostFactory();
        if ("IMAGE".equalsIgnoreCase(type)) return new ImagePostFactory();
        throw new IllegalArgumentException("Unsupported post type: " + type);
    }
}
```

BRIDGE :

FeedView.java

```java
package socialfeed.bridge;

import ...

public abstract class FeedView {  7 usages  1 inheritor  Yermukan Zamira
    protected final FeedRenderer renderer;  2 usages

    protected FeedView(FeedRenderer renderer) {  5 usages  Yermukan Zamira
        if (renderer == null) throw new IllegalArgumentException("Renderer is required");
        this.renderer = renderer;
    }

    public abstract void display(List<Post> posts, FeedSortStrategy strategy);  1 implementation  Yermukan Zamira
}
```

FACADE :

SocialMediaFacade.java

```java
package socialfeed.facade;

import ...
public class SocialMediaFacade {  9 usages  Yermukan Zamira
    private final Feed feed;  4 usages
    private final FeedService service;  4 usages
    private final SocialPlatformFactory platformFactory;  2 usages

    public SocialMediaFacade(SocialPlatformFactory platformFactory) {  10 usages  Yermukan Zamira
        if (platformFactory == null) throw new IllegalArgumentException("Platform factory is required");
        this.platformFactory = platformFactory;
        this.feed = new Feed();
        FeedView view = new TimelineFeedView(platformFactory.createRenderer());
        this.service = new FeedService(feed, view);
    }

    public void follow(String username) {  3 usages  Yermukan Zamira
        feed.subscribe(new UserNotifier(username, platformFactory.createNotificationSender()));
    }

    public void publish(Post post) { service.publish(post); }  5 usages  Yermukan Zamira
    public void showNewestFirst() { service.showFeed(new NewestFirstStrategy()); }  1 usage  Yermukan Zamira
    public void showMostLiked() { service.showFeed(new MostLikedStrategy()); }  1 usage  Yermukan Zamira
    public Feed getFeed() { return feed; }  5 usages  Yermukan Zamira
}

```

DECORATOR :

HashtagDecorator.java

```java
package socialfeed.decorator;

import socialfeed.model.*;
public class HashtagDecorator extends PostDisplayDecorator {  3 usages
    private final String hashtag;  2 usages

    public HashtagDecorator(PostDisplay display, String hashtag) {  4 usages
        super(display);
        this.hashtag = hashtag == null || hashtag.isBlank() ? "" :
                (hashtag.startsWith("#") ? hashtag : "#" + hashtag);
    }

    public String display() { return display.display() + " " + hashtag; }
}

```

STRATEGY :

MostLikedStrategy.java

```java
package socialfeed.strategy;

import ...

public class MostLikedStrategy implements FeedSortStrategy {  3 usages  zika253402
    public List<Post> sort(List<Post> var1) {  2 usages  zika253402
        ArrayList var2 = new ArrayList(var1);
        var2.sort(Comparator.comparingInt(Post::getLikes).reversed());
        return var2;
    }
}

```

VISITOR :

FeedStatisticsVisitor.java

```java
package socialfeed.visitor;

import ...

public class FeedStatisticsVisitor implements PostVisitor {  5 usages  zika253402
    private int textPosts;  2 usages
    private int imagePosts;  2 usages
    private int totalLikes;  3 usages

    public void visit(TextPost var1) {  1 usage  zika253402
        ++this.textPosts;
        this.totalLikes += var1.getLikes();
    }

    public void visit(ImagePost var1) {  1 usage  zika253402
        ++this.imagePosts;
        this.totalLikes += var1.getLikes();
    }

    public int getTextPosts() { return this.textPosts; }

    public int getImagePosts() { return this.imagePosts; }

    public int getTotalLikes() { return this.totalLikes; }
}
```

OBSERVER :

Feed.java

```java
package socialfeed.model;

import ...

public class Feed {  9 usages  tumar
    private final List<Post> posts = new ArrayList<>();  2 usages
    private final List<FeedObserver> observers = new ArrayList<>();  4 usages

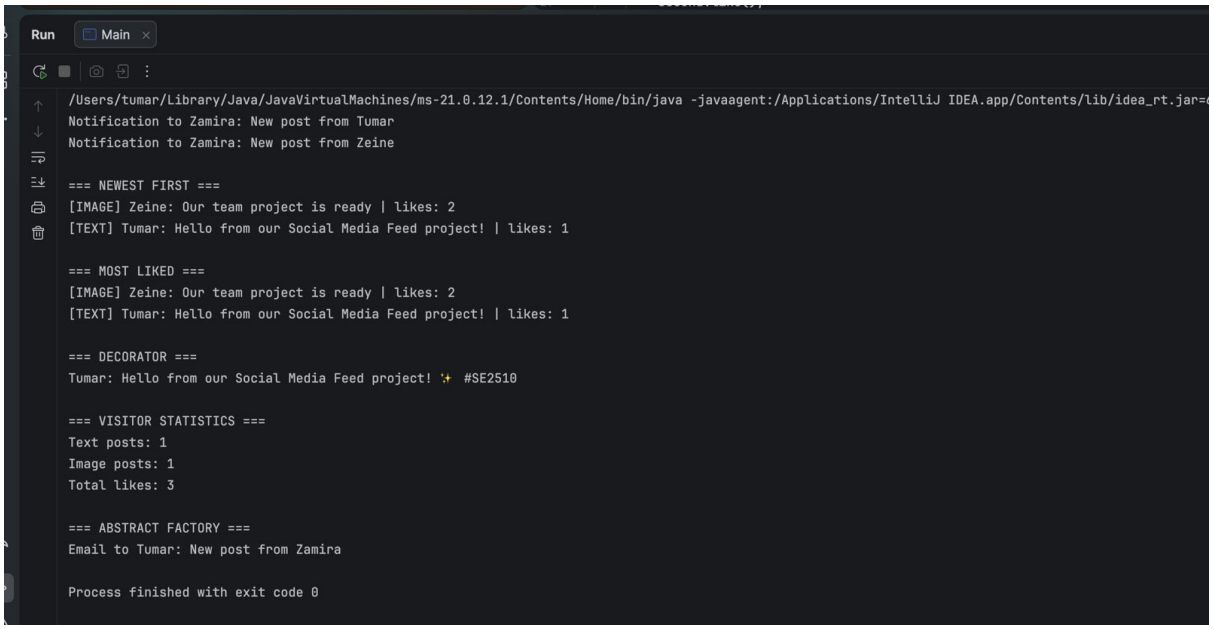
    public void subscribe(FeedObserver observer) {  1 usage  tumar
        if (observer == null) throw new IllegalArgumentException("Observer cannot be null");
        if (!observers.contains(observer)) observers.add(observer);
    }

    public void unsubscribe(FeedObserver observer) { observers.remove(observer); }

    public void publish(Post post) {  1 usage  tumar
        if (post == null) throw new IllegalArgumentException("Post cannot be null");
        posts.add(post);
        for (FeedObserver observer : List.copyOf(observers)) observer.onNewPost(post);
    }

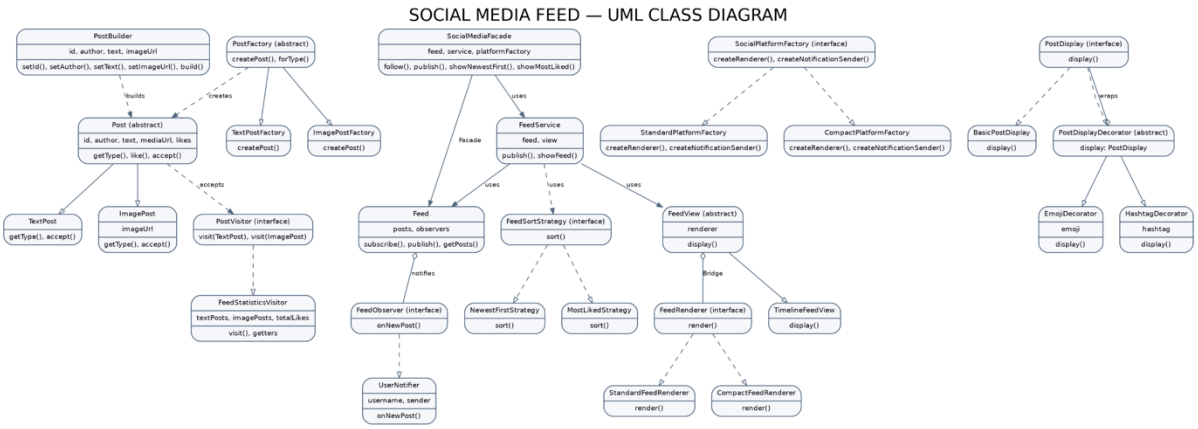
    public List<Post> getPosts() { return Collections.unmodifiableList(posts); }
}
```

APPLICATION SCREENSHOT



UML DIAGRAM

The UML diagram shows the main classes and their relationships in the project.



CONCLUSION :

The project demonstrates nine design patterns in one Java application. Each pattern has a separate responsibility, and the patterns work together in the process of creating, publishing, displaying, and analyzing posts. The application provides a simple example of applying design patterns to a social media feed.

FURTHER WORK :

The application could be extended by adding a database, user accounts, image uploads, a graphical interface, and real notification delivery. Additional sorting and filtering options could also be implemented.